Unit III Assignment

Full Stack
// Khushi Choudhary
// CU240251564

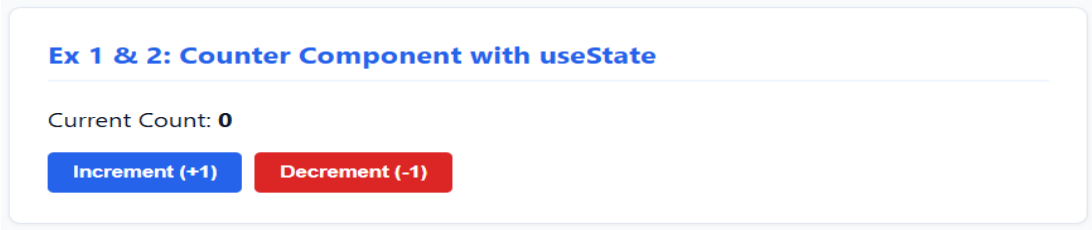
Question 1: Counter Component & Plain Variable Explanation
Functional Counter Component

```
import React, { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(prev => prev + 1)}>Increment</button>
      <button onClick={() => setCount(prev => prev - 1)}>Decrement</button>
    </div>
  );
}
export default Counter;
```

Why Plain Variable Fails to Update UI:
Plain JavaScript variables do not trigger a component re-render when mutated, so React cannot detect the value change to update the DOM.

Question 2: Infinite Re-render Loop Fix
Issue Identification
Calling setCount(count + 1) directly inside the render body executes state updates during render, triggering an infinite loop of re-renders. State updates must be encapsulated within event handlers or useEffect.
Corrected Code

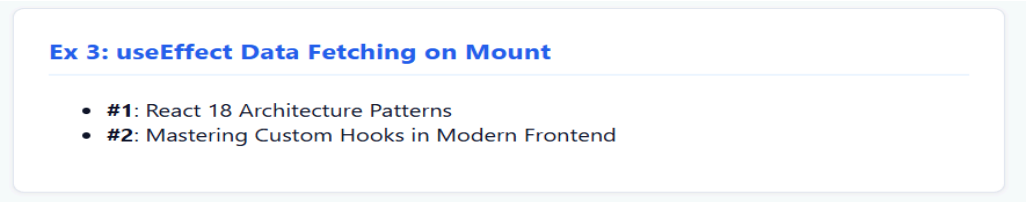
```
import React, { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  const handleIncrement = () => {
    setCount(prev => prev + 1);
  };
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={handleIncrement}>Increment</button>
    </div>
  );
}
```



Question 3: useEffect Mount Fetch & Dependency Array
useEffect Implementation

```
import React, { useState, useEffect } from 'react';
function PostList() {
  const [posts, setPosts] = useState([]);
  useEffect(() => {
    fetch('/api/posts')
```

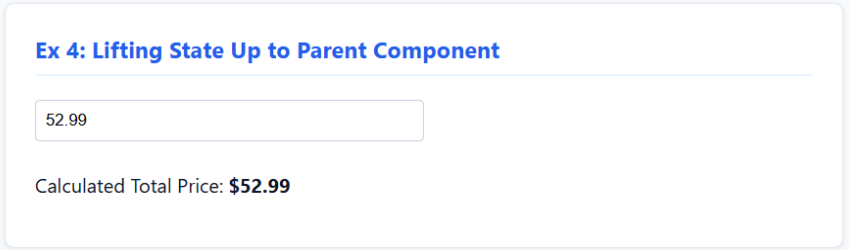
```
      .then(res => res.json())
      .then(data => setPosts(data))
      .catch(err => console.error('Error fetching posts:', err));
  }, []);
  return (
    <ul>
      {posts.map(post => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```



Why Empty Dependency Array [] is Necessary
The empty dependency array [] tells React to run the effect only once when the component initially mounts, preventing infinite API requests on re-render.

Question 4: Lifting State Up to Parent Component
Refactored Code

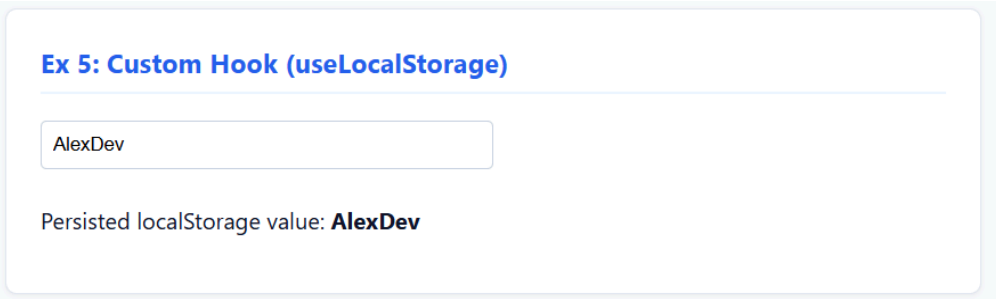
```
import React, { useState } from 'react';
function PriceInput({ price, onPriceChange }) {
  return (
    <input
      type="number"
      value={price}
      onChange={e => onPriceChange(e.target.value)}
    />
  );
}
function PriceDisplay({ price }) {
  return <p>Price: ${price}</p>;
}
// Shared State lifted to Parent Component
function PriceCalculator() {
  const [price, setPrice] = useState(0);
  return (
    <div>
      <PriceInput price={price} onPriceChange={setPrice} />
      <PriceDisplay price={price} />
    </div>
  );
}
```



Question 5: Custom Hook useLocalStorage
Custom Hook Implementation

```
import { useState, useEffect } from 'react';
function useLocalStorage(key, initialValue) {
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) {
      console.error(`Error reading localStorage key "${key}":`, error);
      return initialValue;
    }
  });
  useEffect(() => {
    try {
      window.localStorage.setItem(key, JSON.stringify(storedValue));
    } catch (error) {
      console.error(`Error setting localStorage key "${key}":`, error);
    }
  }, [key, storedValue]);

  return [storedValue, setStoredValue];
}
export default useLocalStorage;
```



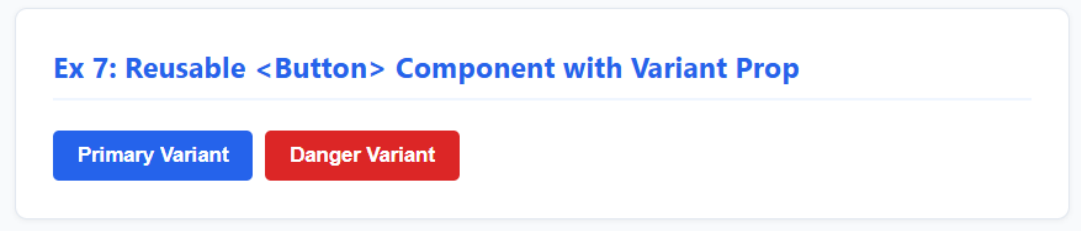
Question 6: React List Key Warning Fix
Corrected Code

```
{items.map(item => (
  <li key={item.id}>{item.name}</li>
))}
```

Question 7: Reusable <Button> Component with Variant Prop
Component Code

```
import React from 'react';
function Button({ variant = 'primary', children, onClick, ...rest }) {
  const buttonClass = variant === 'danger' ? 'btn btn-danger' : 'btn
btn-primary';

  return (
    <button className={buttonClass} onClick={onClick} {...rest}>
      {children}
    </button>
  );
}
export default Button;
```

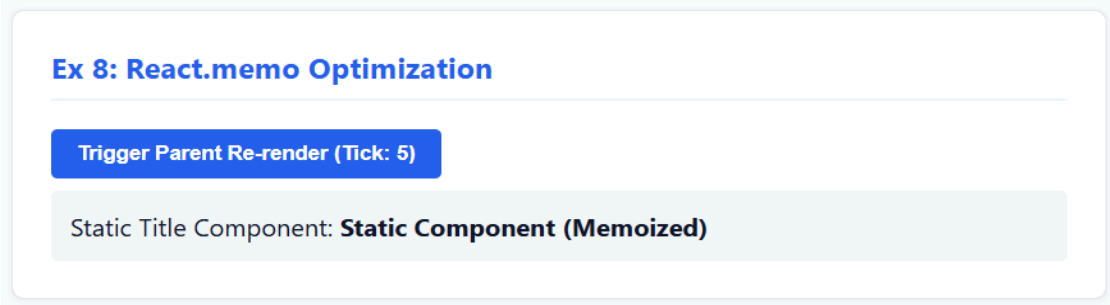


Question 8: Preventing Child Re-renders with React.memo
Explanation

By default in React, updating a parent's state forces all of its child components to re-render, even if the child's own props remain unchanged.

Corrected Code

```
import React, { useState, memo } from 'react';
function ParentDashboard() {
  const [tick, setTick] = useState(0);
  return (
    <div>
      <button onClick={() => setTick(tick + 1)}>Refresh ({tick})</button>
      <ChildCard title="Static Title" />
    </div>
  );
}
const ChildCard = memo(function ChildCard({ title }) {
  console.log('ChildCard rendered');
  return <h3>{title}</h3>;
});
```



Question 9: React Router v6 Configuration & Links
Code Implementation

```
import React from 'react';
import { BrowserRouter, Routes, Route, Link, useParams } from
'react-router-dom';
function Home() { return <h2>Home Page</h2>; }
function ProductList() { return <h2>Product List</h2>; }
function ProductDetail() {
  const { id } = useParams();
  return <h2>Product Details for ID: {id}</h2>;
}
function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> |
        <Link to="/products">Products</Link> |
```

```
        <Link to="/products/42">Product #42</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/products" element={<ProductList />} />
        <Route path="/products/:id" element={<ProductDetail />} />
      </Routes>
    </BrowserRouter>
  );
}
export default App;
```

Question 10: Consolidated Form State Object & Handler
Refactored Code

```
import React, { useState } from 'react';
function UserForm() {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    phone: ''
  });
  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: value
    }));
  };
  return (
    <form>
      <input name="name" value={formData.name} onChange={handleChange}
placeholder="Name" />
      <input name="email" value={formData.email} onChange={handleChange}
placeholder="Email" />
      <input name="phone" value={formData.phone} onChange={handleChange}
placeholder="Phone" />
    </form>
  );
}
```

